

AIRA-OS: Autonomous Agentic Operating System

Complete Technical Architecture, Codebase Specification & SIH Presentation Guide

AIRA-OS is an autonomous, cross-device personal AI Operating System bridging an Android mobile device and a Windows laptop into a unified intelligent environment. Unlike traditional reactive chatbots that merely generate text responses, AIRA decomposes high-level goals into multi-step execution plans, grounds visual UI elements via multimodal vision, enforces human-in-the-loop safety guardrails, and continuously calibrates its behavior via adaptive learning.

1. Three-Tier System Architecture

The system operates across three tightly integrated layers communicating over secure local network protocols and cloud API fallback chains:

Layer	Core Technologies	Primary Responsibilities
1. Mobile Client (Android)	Flutter, Dart, Riverpod, MethodChannels, SpeechToText, FlutterTTS	User interface, live plan execution streaming, voice interaction, Android hardware controls (alarms, torch, calls, app launcher).
2. Desktop Agent (Windows)	Python 3.10+, FastAPI, Uvicorn, Win32 ctypes, PyAutoGUI, MSS	Zero-latency mouse trackpad acceleration, multimodal visual screen grounding, terminal execution, file system management.
3. Intelligence Brain (AI & Cloud)	Groq (Llama-3), Gemini 1.5, OpenRouter, SharedPreferences, Supabase pgvector	Goal planning, self-check reflection critic, guardrail action tiers, adaptive memory learning, and multi-agent role orchestration.

2. Complete Codebase Structure & File Inventory

Below is the complete structural breakdown of all directories and key functional modules in the AIRA repository:

Path / Module	Component Role & Implementation Details
lib/core/agent/goal_planner_engine.dart	Decomposes natural language user goals into structured JSON ReAct subtask graphs. Assigns tool mappings and execution tiers.
lib/core/agent/self_check_reflector.dart	Pre-action reflection critic. Scans generated drafts for corporate filler, placeholder brackets, and tone inaccuracies. Persists audit logs.

<code>lib/core/agent/action_guardrail_manager.dart</code>	Enforces safety tiers (Free-Run vs. Approval-Required). Halts execution for sensitive actions (emails, SMS, calls, deletions) until approved.
<code>lib/core/agent/agent_tool_registry.dart</code>	Dynamic tool registry with JSON schemas. Auto-selects between live web search, native alarms, n8n workflows, and laptop actions.
<code>lib/core/agent/adaptive_outcome_learner.dart</code>	Monitors user edits and approval decisions. Dynamically injects learned brevity and style rules into system prompts.
<code>lib/core/agent/agent_orchestrator.dart</code>	Multi-agent swarm router delegating across SchedulerSubAgent, EmailCommsSubAgent, MemoryLearningSubAgent, and AutomationDeviceSubAgent.
<code>lib/features/chat/presentation/widgets/</code>	Contains PlanExecutionCard (live animated progress card), ActionApprovalCard (interactive approval modal), and MessageBubble.
<code>aira_desktop/vision_agent.py</code>	Multimodal screen grounding using Groq Llama-3.2-11B-Vision. Calculates exact (x, y) coordinates of UI elements and verifies screen state.
<code>aira_desktop/agent_runner.py</code>	Autonomous self-healing ReAct action loop on Windows. Diagnoses execution failures and generates dynamic recovery steps.
<code>aira_desktop/main.py</code>	FastAPI server hosting endpoints for trackpad ctypes acceleration, clipboard synchronization, terminal shell execution, and PIN auth.

3. The 6-Phase Agentic Execution Pipeline

AIRA's intelligence core operates through a rigorous 6-phase sequential pipeline designed to maximize autonomy while guaranteeing safety:

Phase 1: Goal Planning Layer (ReAct Decomposition)

Accepts an unstructured, high-level goal (e.g., 'Handle my morning'). Instead of executing blindly, it prompts the LLM to return an ordered dependency graph of subtasks with assigned tools and safety tiers. Streams live state updates to the UI.

Phase 2: Reflection & Self-Check Loop

Acts as an internal critic. Before sensitive communications (emails, SMS, messages) are dispatched, the reflector scans the draft for robotic filler phrases ('I hope this finds you well'), unresolved placeholder brackets ('[Insert Date]'), and tone mismatches. Emits a refined draft and logs the audit record.

Phase 3: Human-in-the-Loop Guardrail Tiers

Strictly partitions actions into two categories:

- Free-Run Tier: Safe queries, calendar reads, note creations, alarms, web searches.
- Approval-Required Tier: External emails, phone calls, SMS messages, file deletions, payment triggers.

For approval actions, execution pauses and renders an interactive card with Approve, Edit, and Reject buttons.

Phase 4: Tool Registry & Dynamic Auto-Selection

Replaces brittle regex keywords with a schema-based tool dispatcher. Matches subtask intent against tool descriptions to invoke web search, native Android services, laptop automation, or external n8n workflows automatically.

Phase 5: Adaptive Memory & Outcome Learning

Tracks the history of user decisions (accepted, edited, rejected). When a user repeatedly edits drafts (e.g., trimming long emails), the engine computes character deltas and injects persistent XML directives (<adaptive_learned_preferences>) into future prompts.

Phase 6: Multi-Agent Role Split & Swarm Orchestrator

Decomposes complex multi-domain instructions across specialized sub-agents: SchedulerSubAgent (agenda & deadlines), EmailCommsSubAgent (communication & drafting), MemoryLearningSubAgent (preference tracking), and AutomationDeviceSubAgent (hardware execution).

4. Technical Glossary & Key Concepts Dictionary

Essential technical terms used across AIRA-OS that every team member should know and explain at hackathons:

Term	Definition & Context in AIRA-OS
------	---------------------------------

Agentic AI	An AI system capable of autonomous goal pursuit, multi-step planning, tool usage, environment perception, and self-correction, rather than simple single-turn conversation.
ReAct Pattern	Reason + Act: A prompting paradigm where the LLM interleaves chain-of-thought reasoning with tool execution actions to solve complex multi-step problems.
MethodChannel	The native communication bridge in Flutter that allows Dart code to invoke platform-specific Java/Kotlin methods on Android (e.g., hardware camera, flashlight, alarms).
Win32 ctypes	Python foreign function library used to call Windows C-kernel APIs (user32.dll) directly, enabling 0-millisecond trackpad cursor movements without overhead.
Visual Grounding	The process of taking a screen image, parsing UI elements through a multimodal Vision LLM, and predicting normalized (x, y) coordinates to physically click buttons.
Human-in-the-Loop	A safety architecture where high-risk AI actions are halted in a pending state until a human user explicitly reviews and approves the execution.
State Management (Riverpod)	A reactive, compile-safe state management framework for Flutter that decouples business logic from UI widgets and manages data streams cleanly.
LLM Fallback Chain	A resilient multi-provider pipeline that routes requests from Primary (Groq) to Fallback 1 (Gemini) to Fallback 2 (OpenRouter) to prevent system downtime.

5. Smart India Hackathon (SIH) Presentation Guide

Strategic presentation flow designed to captivate judges within 3 minutes and demonstrate clear novelty:

A. The 60-Second Opening Pitch

*"Most AI tools today are reactive chatbots that only output text, while traditional voice assistants like Siri are rigid command executors. We built **AIRA-OS** — a unified Agentic Operating System connecting your phone and laptop into a single autonomous mind. You give one goal, AIRA decomposes the plan, sees your laptop screen, executes native actions, self-checks for errors, guards sensitive actions with approval modals, and learns your preferences over time."*

B. 3-Minute Live Demonstration Flow

- **Step 1 — Hands-Free Trigger & Laptop Connection:** Open AIRA, show the TickTick agenda pill and live date in the AppBar. Speak 'Hey AIRA' or tap the AppBar mic.
- **Step 2 — Goal Decomposition & Live Card:** Say 'Handle my morning: check my agenda, time-block my day, and check tech headlines.' Point to the live PlanExecutionCard showing animated step spinners and checkmarks.
- **Step 3 — Self-Check & Approval Guardrail:** Say 'Draft an email to the team regarding our submission deadline.' Show how the reflection engine cleans placeholders and **pauses at the interactive Action Approval Card** awaiting your tap.
- **Step 4 — Laptop Vision & Oms Trackpad:** Open Laptop Remote, demonstrate seamless Oms cursor glide via Win32 ctypes, and run an autonomous laptop task ('Open YouTube and search hackathon projects').

C. Judge Q&A; Defense Matrix

Potential Judge Question	Definitive Technical Answer
Q: How do you prevent hallucinated or dangerous actions?	A: We implemented a two-tier guardrail architecture (ActionGuardrailManager). Read-only actions run free, while write/send/delete actions are held in a pending state. Our Self-Check Reflector verifies accuracy before presenting an interactive approval card.
Q: Why not use existing cloud assistants?	A: Commercial assistants cannot see your computer screen, run local terminal scripts, or time-block academic schedules dynamically. AIRA provides true multimodal computer use and cross-device symbiosis over local Wi-Fi with zero external cloud latency.
Q: How does AIRA adapt to user habits?	A: Our Adaptive Outcome Learner logs decisions (accept/edit/reject) and computes length/tone deltas. Over time, it injects calibrated behavioral directives into the personality system prompt.